Programmazione II

Prototipo di una funzione.

Il prototipo di una funzione costituisce una dichiarazione della funzione, e come tale fornisce al compilatore le informazioni necessarie a gestire la funzione stessa

Variabili globali, statiche, esterne e costanti.

- Una variabile dichiarata **global** è accessibile da qualunque punto del programma.
- Una variabile dichiarata **static** non viene distrutta al termine della funzione contenitrice.
- Una variabile dichiarata **extern** indica che la sua dichiarazione si trova altrove.
- Una variabile dichiarata **const** è una costante.

Funzioni inline e macro

Ambo due dichiarazioni hanno lo stesso comportamento: una volta richiamate, esse vengono sostituite con il corpo stesso della funzione o della macro.

La differenza risiede nella interpretazione da parte del compilatore: mentre le funzioni inline vengono interpretate come “normali” funzioni, le macro vengono direttamente sostituite dal preprocessore senza controllarne la sintassi/correttezza.

Inizializzazione di una array con complessità logaritmica.

È possibile inizializzare un array in tempo logaritmico utilizzando la funzione memcpy, che prende come parametri: destinazione, sorgente e dimensione in byte. L’inizializzazione avverrebbe tramite la copia di porzioni di memoria.

```
template <class T> void init(T* v, int len, T value) {
    v[0] = value;
    int size = 1;
    while(size * 2 < len) {
        memcpy(v + size, v. size * 4);
        size *= 2;
    }
    memcpy(array+size, v, (len-size) * 4);
}
```

Array di funzioni.

Essendo una funzione un puntatore al proprio corpo, è possibile creare un array di puntatori a funzioni.

Si ipotizzino 4 funzioni: somma, quoziente, prodotto, differenza, con lo stesso tipo di ritorno int.

È possibile creare un array int (*calcoli)(int) { somma, quoziente, prodotto, differenza };

Funzioni statiche e costanti nella OOP.

Nella OOP :

- Una funzione statica non è legata alla singola istanza.
- Una funzione costante non può modificare lo stato dell'istanza.

Complessità e analisi asintotica

Lo studio della complessità di un algoritmo si dirama in:

- **Studio della complessità spaziale**: quantità di risorse utilizzate;
- **Studio della complessità temporale**: tempo di esecuzione;

Essendo la prima legata all'avanzamento tecnologico dei dispositivi, ci focalizzeremo sulla seconda:

La complessità temporale cresce al crescere della dimensione n dell'input, quindi dipende dalla quantità di dati.

Lo strumento utilizzato per analizzare la complessità è l'**analisi asintotica**, ed essendo quest'ultima uno strumento matematico, bisogna mutare il tempo di esecuzione in una funzione T(n) in funzione della dimensione dei dati n.

In genere la funzione T(n) quantifica il numero di comando effettuati dall'algoritmo.

Data una istanza di dimensione n, nel **caso peggiore**, l'algoritmo ha una complessità temporale

$$T(n) = O(f(n))$$

Dove f(n) è il limite superiore del tempo di esecuzione dell'algoritmo nel caso pessimo.

È importante utilizzare il caso peggiore poiché è quello che misura meglio l'efficienza dal punto di vista tecnico (vedasi bogosort).

Algoritmi di ordinamento

Caratteristiche degli algoritmi di ordinamento.

Un algoritmo di ordinamento si dice:

- **Stabile** se preserva l'ordine relativo dei dati aventi analoga chiave.
- **In-place** se durante il processo di ordinamento non abbisogna di allocare ulteriore memoria.
- **Adattivo** se varia il suo andamento a seconda delle circostanze, ovvero se esegue diverse sequenze di operazioni in funzione del risultato dei confronti

Bubblesort

Il bubblesort è un algoritmo di ordinamento in-place basato sul confronto a due a due.

Il caso peggiore, ovvero quando gli elementi sono ordinati in modo decrescente, è $O(n^2)$, quindi lo è anche la sua complessità. Il caso migliore, ovvero quando gli elementi sono già ordinati, è O(n).

```
template <class T> void bubblesort(T* v, int len) {

    for(int i = len; i >= 1; i--) {
        for(int j = 0; j < i - 1; j++) {
            if(v[j] > v[j+1]) {
                T tmp = v[j];
                v[j] = v[j+1];
                v[j+1] = tmp;
            }
        }
    }
}
```

Selectionsort

Il Selection-sort, in italiano ordinamento per selezione, è un algoritmo relativamente semplice per ordinare un array che opera in place. L'algoritmo è di tipo non adattivo, ossia il suo tempo di esecuzione non dipende dall'input ma dalla dimensione dell'array. L'algoritmo seleziona di volta in volta il numero minore nella sequenza di partenza e lo sposta nella sequenza ordinata; di fatto la sequenza viene suddivisa in due parti: la sottosequenza ordinata, che occupa le prime posizioni dell'array, e la sottosequenza da ordinare, che costituisce la parte restante dell'array.

La sua complessità è $O(n^2)$ nel caso peggiore.

```
template <class T> void selectionsort(T *v, int len) {
    for(int i = 0; i < len; i++) {
        int min = i;
        for(int j = i; j < len; j++) {
            if( v[min] > v[j] ) min = j;
        }
        T tmp = v[i];
        v[i] = v[min];
        v[min] = tmp;
    }
}
```

Insertionsort

L'Insertion sort, in italiano ordinamento a inserimento, è un algoritmo relativamente semplice per ordinare un array. Non è molto diverso dal modo in cui un essere umano, spesso, ordina un mazzo di carte. Esso è un algoritmo in place, cioè ordina l'array senza doverne creare una copia, risparmiando memoria. Pur essendo molto meno efficiente di algoritmi più avanzati, può avere alcuni vantaggi: ad esempio, è semplice da implementare ed è efficiente per insiemi di partenza che sono quasi ordinati.

La sua complessità è $O(n^2)$, mentre nel caso migliore è $O(n)$.

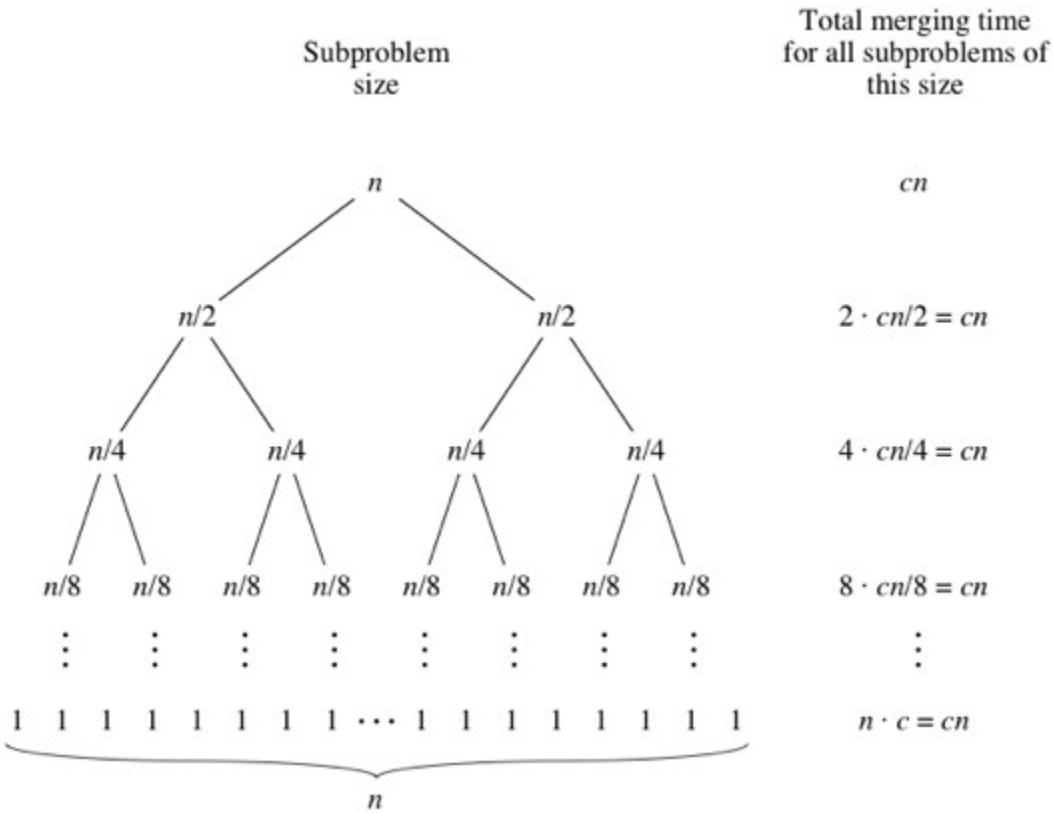
```
template <class T> void insertionsort(T *v, int len) {

    for(int i = 1; i < len; i++) {
        int j = i;
        while( j > 0 && v[j] < v[j-1]) {
            T tmp = v[j];
            v[j] = v[j-1];
            v[j-1] = tmp;
            j--;
        }
    }
}
```

Mergesort

Il merge sort è un algoritmo di ordinamento basato su confronti che utilizza un processo di risoluzione ricorsivo, sfruttando la tecnica del Divide et Impera, che consiste nella suddivisione del problema in sottoproblemi della stessa natura di dimensione via via più piccola. Fu inventato da John von Neumann nel 1945.
Concettualmente, l'algoritmo funziona nel seguente modo:
- Se la sequenza da ordinare ha lunghezza 0 oppure 1, è già ordinata e nulla viene fatto. Altrimenti:
- La sequenza viene divisa (divide) in due metà (se la sequenza contiene un numero dispari di elementi, viene divisa in due sotto sequenze di cui la prima ha un elemento in più della seconda);
- Ognuna di queste sotto-sequenze viene ordinata, applicando ricorsivamente l'algoritmo (impera);
- Le due sotto-sequenze ordinate vengono fuse (combina). Per fare questo, si estrae ripetutamente il minimo delle due sotto-sequenze e lo si pone nella sequenza in uscita, che risulterà ordinata.

Il mergesort è generalmente sviluppato con il metodo top-down e non in-place, ma può essere sviluppato in-place tramite la tecnica bottom-up, dove ogni valore dell’array in input viene trattato come un array di dimensione 1.

Sia nel caso peggiore che nel caso migliore l’algoritmo ha una complessita di $O(n*\log n)$.



Implementazione:

```
template <class T> void merge(T *v, int len, int mid) {
    int i = 0;
    int j = mid;
    int h = 0;

    T w[len];

    while(i < mid && j < len) {
        if(v[i] < v[j]) w[h++] = v[i++];
        else w[h++] = v[j++];
    }

    if(i < mid) for(int k = i; k < mid; k++) w[h++] = v[k];

    else for(int k = j; k < len; k++) w[h++] = v[k];

    for(int i = 0; i < len; i++) v[i] = w[i];
}

template <class T> void mergesort(T* v, int len) {
    if(len > 1) {
        int mid = len / 2 + (len % 2);
        mergesort( v, mid );
        mergesort( v + mid, len - mid );
        merge(v, len, mid);
    }
}
```

Quicksort

Il quicksort è un algoritmo di ordinamento basato su confronti che utilizza un processo di risoluzione ricorsivo, anche esso basato sulla tecnica **Divite et Impera.**

Concettualmente il quicksort segue il seguente procedimento:

- Si effettua un ordinamento parziale scegliendo un elemento qualsiasi della sequenza, detto **pivot,** e spostando tutti gli elementi minori a sinistra, mentre quelli maggiori a destra, senza tenere conto dell'ordine. Alla fine degli spostamenti il pivot si troverà nella sua posizione definitiva.
- Si applica ricorsivamente l'ordinamento parziale alle due sottosequenze formatosi ai lati del pivot.
- Una sottosequenza di lunghezza pari a 0 o 1 è già ordinata.

Caratteristiche:

L'algoritmo può essere suddiviso in due parti:

- Un processo detto **partition** che si occupa di individuare il pivot e effettuare gli spostamenti per ottenere le due sottosequenze;
- Un processo detto **quicksort** che viene richiamato ricorsivamente sulle due sottosequenze dopo il processo partition.

È una algoritmo in-place, in quanto per ordinare opera direttamente sull'array.

Nel caso peggiore, ovvero un array già ordinato, la complessità è $O(n^2)$, ma nel caso medio risulta essere più efficiente di altri algoritmi (come il mergesort).

Una particolare implementazione del processo partition può influenzare la complessità, alcuni partizionamenti noti sono: casuale, non casuale, metodo della mediana.

```
template <class T>
int partition(T v[], int top, int bottom) {
    T x = v[top];
    int i = top - 1;
    int j = bottom + 1;
    do {
        do j--;
        while (x < v[j]);

        do i++;
        while (x > v[i]);

        if (i < j) scambia(v, i, j);
    } while (i < j);
    return j;
}

template <class T>
void quicksort(T v[], int top, int bottom) {
    int middle;
    if (top < bottom) {
        middle = partition(v, top, bottom);
        quicksort(v, top, middle);          // sort top partition
        quicksort(v, middle+1, bottom);     // sort bottom partition
    }
    return;
}
```

Natural merge.

Il natural merge è un algoritmo utilizzato per ottenere un unico array ordinato dall'unione di due array a loro volta ordinati di dimensione variabile.

```
int* naturalMerge( int v[], int w[], int lenv, int lenw ) {

    int* s = new int[lenv + lenw];
    int i = 0, j = 0, h = 0;

    while(i < lenv && j < lenw) {
        if(v[i] < w[j]) s[h++] = v[i++];
        else s[h++] = w[j++];
    }

    if(i < lenv)
        for(int k = i; k < lenv; k++) s[h++] = v[k];
    else
        for(int k = j; k < lenw; k++) s[h++] = w[k];

    return s;
}
```

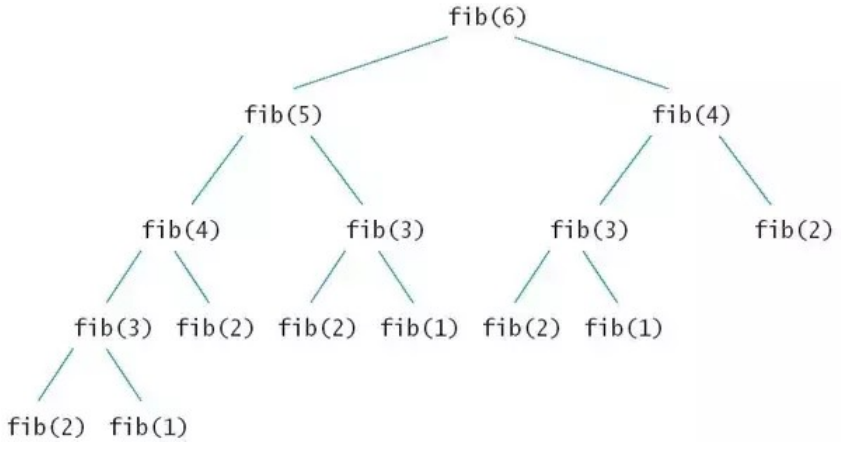
Algoritmo di Fibonacci.

La successione di Fibonacci (detta anche successione aurea), in matematica indica una successione di numeri interi in cui ciascun numero è la somma dei due precedenti.

È possibile formulare l'algoritmo ricorsivamente come segue:

```
int fibonacci(int n) {
    if(n <= 2) return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Fibonacci ha una complessità $O(n^2)$.



La torre di Hanoi.

La torre di hanoi è un rompicapo matematico composto da tre paletti ed n dischi di grandezza crescente. Esso consiste nello spostare i dischi dalla prima torre alla terza torre secondo i seguenti vincoli:

- Si può spostare un solo disco per volta;
- Un disco non può essere posizionato sopra un'altro di dimensione minore.

Si può dimostrare che dati n dischi, il numero di mosse necessarie per risolvere il rompicapo corrispondono ad $2^n - 1$. La complessità di una possibile implementazione ricorsiva è quindi $O(2^n)$.

```
void hanoi(int n, char A, char B, char C) {

  if(n==1){
    cout << "From " << A << " to " << C << endl;
    return;
  }

  hanoi(n-1,A,C,B);
  hanoi(1,A,B,C);
  hanoi(n-1,B,A,C);
}
```

Ricerca ricorsiva di un elemento di un array in tempo lineare [O(n)].

```
int find(int v[], int len, int key)
{
    if(len == 0) return -1;
    if(*v == key) return *v;
    find( v+1, len-1, key );
}
```

Permutazioni.

```
void permutazioni(char *c, int dim, int j) {
    if(j == dim-1) {
        for(int i = 0; i < dim; i++)
            cout << c[i];
        cout << endl;
    }
    for(int i = j; i < dim; i++) {
        char tmp = c[i];
        c[i] = c[j];
        c[j] = tmp;
        permutazioni( c, dim, j+1 );
        tmp = c[i];
        c[i] = c[j];
        c[j] = tmp;
    }
}
```

Named constructors: c++ ed i multi costruttori.

In c++ non è possibile distinguere l'overloading del costruttore con nomi differenti, ma nel caso di necessità vi sono alcune soluzioni plausibili: si potrebbe definire privatamente il costruttore e utilizzare metodi statici per l'inizializzazione. Di seguito viene riportato un esempio:

```
class Rectangle {

    private:

    int base, heigth;

    Rectangle( int base, int heigth ) {
        this->base = base;
        this->heigth = heigth;
    }

    public:

        static Rectangle rectangle( int base, int heigth ) {
            Rectangle *tmp = new Rectangle( base, heigth );
            return *tmp;
        }

        static Rectangle square( int side ) {
            Rectangle *tmp = new Rectangle( side, side );
            return *tmp;
        }

};
```

Immutable objetcs & singleton.

Durante lo sviluppo può nascere la necessità di creare un oggetto con pattern singleton, ovvero un oggetto instanziabile una singola volta, oppure un oggetto immutabile, il cui stato resta invariato durante l'esecuzione del programma.

Nel primo caso, la soluzione più comune è quella di utilizzare un metodo statico che ritorna la singola instanza.

Nel secondo caso, si può fare a meno di metodi set, fornendo solo metodi get e facendo attenzione a tornare copie di sicurezza.

Strutture dati

List

Una lista è una struttura dati dinamica, ovvero non ha una dimensione prefissata. Gli elementi contenuti prendono il nome di nodi e possono trovarsi in porzioni di memoria non contigue.

Se ogni nodo è collegato al successivo, la lista prende il nome di **linked-list**.

Se oltre al successivo, il nodo è collegato anche al precedente, allora la lista prende il nome di **double-linked-list**.

Se il primo nodo della lista è collegato all'ultimo, allora la lista è **circolare**.

Per facilitare l'implementazione della lista è solito l'utilizzo delle sentinelle **header** e **trailer,** ovvero nodi vuoti posizionati rispettivamente all'inizio ed alla fine della lista.

In questo modo l'accesso ad uno specifico elemento parte dal nodo successivo all'**header** e scorre finché il successivo non risulta essere il **trailer**.

La **complessità dell'accesso** ad un elemento nel caso peggiore è O(n).

L'inserimento nella lista può essere eseguito in testa, in coda o in ordine.

Nel primo e nel secondo caso, un nuovo nodo viene banalmente inserito successivamente all'header o precedentemente al trailer. La complessità risulta essere costante.

Nel terzo caso, ovvero l'inserimento ordinato, l'elemento scorre la lista e si posiziona tra due nodi ordinatamente in base al suo valore. Nel caso peggiore, ovvero l'elemento inserito è il più grande, la complessità risulta essere O(n).

La cancellazione necessità della ricerca dell'eventuale nodo x, quindi complessità O(n), ed una volta trovato lo espelle dalla lista collegando il precedente di x con il successivo di x e viceversa.

Stack

Una Pila (in inglese Stack) è una struttura dati di tipo LIFO (Last In First Out). L'operazione di inserimento in una Pila è detta "push", mentre l'operazione di cancellazione dalla struttura è detta "pop".

Concettualmente lo stack è una sequenza di nodi. Esso contiene il riferimento alla "head" ovvero il nodo inserito per ultimo e alla quantità dei nodi inseriti.

I due metodi principali sono:

- **Push**, ovvero l'inserimento in testa. Se lo stack è vuoto allora il nodo inserito diventa la "head", altrimenti la vecchia "head" viene prima collegata al nuovo nodo e dopo aggiornata puntando a quest'ultimo.
- **Pop**, ovvero l'estrazione dalla testa. Il metodo ritorna il valore del nodo "head" e aggiorna la "head" al precedente del nodo puntato, escludendo il nodo estratto.

Queue

Una Coda (in inglese Queue) è una struttura dati di tipo FIFO (First In First Out). L'operazione di inserimento in una Pila è detta "enqueue", mentre l'operazione di cancellazione dalla struttura è detta "dequeue".

La coda viene implementata tenendo conto della testa e della, appunto, coda.

Inizialmente la testa e la coda si puntano a vicenda.

Durante l'enqueue il nuovo nodo viene inserito in coda. Se risulta essere il primo inserimento allora la testa punterà al nuovo nodo.

Quando si effetta un dequeue, la Queue ritorna il valore del nodo x in testa, preleva il precedente di x e aggiorna la testa con il nodo prelevato, escludendo il nodo x.

Alberi

Un albero è una collezione di nodi e di archi, per cui:

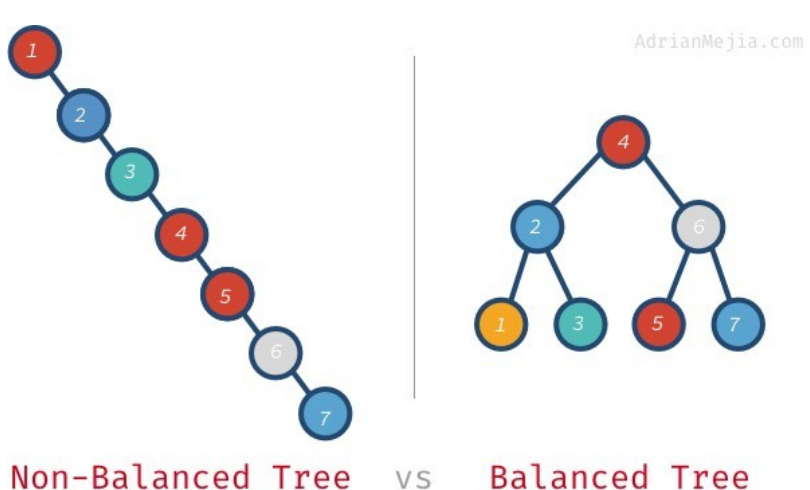
- *Ogni nodo, eccetto uno, detto **radice**, ha solo un predecessore e 0 o più successori, la radice non ha predecessori.*
- *Esiste **un unico cammino** dalla radice ad ogni altro nodo.*
- *Ogni nodo contiene un valore.*
- *Un nodo che non ha successori è detto foglia.*
- *Il padre di un nodo n è il nodo immediatamente prima di n nel cammino verso la radice.*
- *Un antenato di un nodo n è un qualsiasi nodo presente nel cammino di n verso la radice.*
- *Due nodi sono **fratelli** se hanno il nodo padre in comune.*
- *La **profondità** di un nodo è la lunghezza del cammino radice-nodo.*
- *L' **altezza** di un albero corrisponde alla distanza tra la radice e la foglia più distante.*
- *Il **grado** di un albero è determinato dal massimo numero di figli che può avere un nodo.*

Un albero è **posizionale** se la posizione è specifica per ogni figlio.

Un albero binario è **completo** se tutte le foglie sono allo stesso livello ed ogni livello è pieno.

Osservazione: Se un albero b-ario completo ha le foglie all'n-esimo livello, allora $h = \log_b (n)$

Ipotizzando di sapere solamente il numero n di nodi di un albero e nient'altro allora l'altezza h è compresa tra il caso migliore (albero completo) h = floor(log (n)) ed il caso peggiore h = n; (floor = approssimato)



<u>BST</u> [Binary Search Tree]

Un BST è un albero binario posizionale regolato dal seguente protocollo:

Sia x un generico nodo del BST:

- Il sottoalbero che ha come radice il figlio sinistro di x conterrà solo valori minori o uguali alla chiave di x.
- Il sottoalbero che ha come radice il figlio destro di x conterrà solo valori maggiori alla chiave di x.

Un albero **bilanciato** ha quantità quantomeno simili di nodi a destra e a sinistra.

Il più **piccolo** elemento starà nel nodo più a sinistra, mentre quello più **grande** starà nel nodo più a destra.

Riempire un BST vuoto.

Per riempire un BST vuoto bisogna adottare una specifica tecnica che prevede l'inserimento degli elementi in ordine crescente partendo dal nodo più a sinistra, muovendosi verso destra.

Cercare un **successore** di un nodo x in un BST equivale a:

- Cercare il minimo nel sottoalbero destro.
- Se non esiste il sottoalbero destro, risalire verso gli ancestor fino al primo antenato che ha una chiave maggiore o uguale a quella di x.

Simmetricamente per il **predecessore.**

Visita del Binary Search Tree.

Vi sono 3 famosi tipi di visita nel BST, tutti con complessità O(n), e sono i seguenti:

```
void _inorder(Node<H>* r) {
  if(r) {
    _inorder(r->getLeft());
    cout << *(r->getKey()) << " ";
    _inorder(r->getRight());
  }
}

void _postorder(Node<H>* r) {
  if(r) {
    _postorder(r->getLeft());
    _postorder(r->getRight());
    cout << *(r->getKey()) << " ";
  }
}

void _preorder(Node<H>* r) {
  if(r) {
    cout << *(r->getKey()) << " ";
    _preorder(r->getLeft());
    _preorder(r->getRight());
  }
}
```

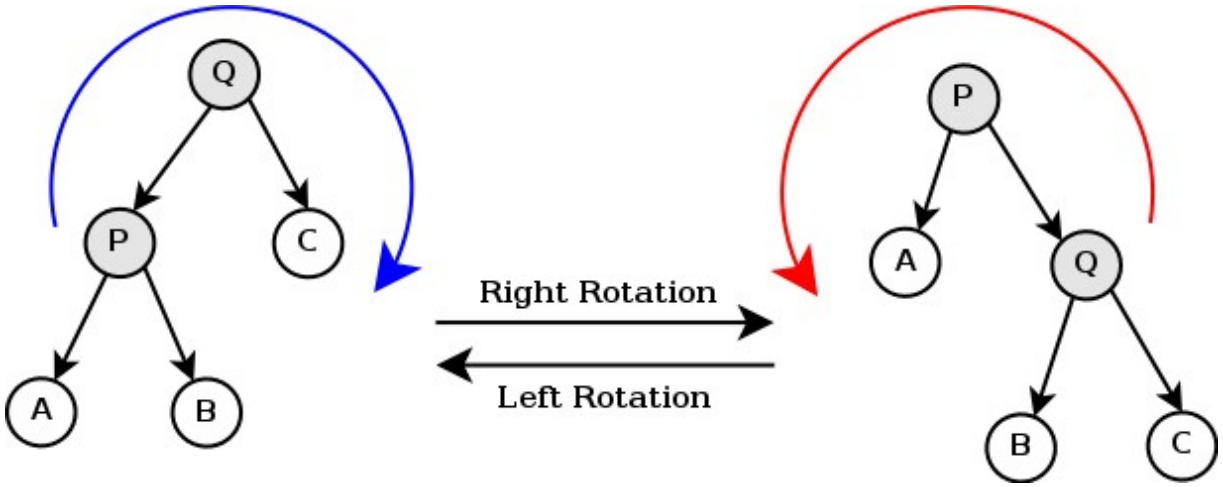
Cancellazione.

La cancellazione di un nodo nel BST si dirama in 3 casi:

- Il nodo è una foglia => si cancella banalmente.
- Il nodo ha solo un un figlio => si cancella il nodo corrente e si collega il figlio al parent.
- Il nodo ha due figli => si sostituisce il nodo con il successore e si cancella quest'ultimo.

La complessità della cancellazione è O(h).

Rotazione.



Ultime definizioni:

Definiamo **peso** di un nodo x la somma delle chiavi memorizzate all'interno del sottoalbero radicato in x.

Grafi

Un grafo è un insieme di elementi detti nodi o vertici che possono essere collegati fra loro da archi. Più formalmente, si dice grafo una coppia ordinata G=(V, E) di insiemi, con V insieme dei nodi ed E insieme degli archi, tali che gli elementi di E siano coppie di elementi di V.

Quando gli archi sono direzionati si parla di Grafo ***diretto*** *o* ***orientato*** *e gli elementi di E sono delle coppie ordinate di elementi di V. Se il nodo u è connesso al nodo v attraverso un arco, allora si dice che v è* ***adiacente*** *ad u.*

Se gli archi non sono direzionati allora si parla di grafo ***non orientato****, quindi inserire un arco tra un nodo A ed un nodo B permetterà la transizione in entrambi i versi.*

Sia Vi un nodo del grado, allora un nodo Vj è adiacente a Vi <=> Esiste un arco (Vi, Vj)

Quindi: Adj(V) = { U : (V, U) ∈ E }

Il **grado** di un nodo V corrisponde alla cardinalità dell'insieme dei nodi adiacenti a V. Formalmente:

grado(V) = | adj(V) |

Il cammino il un grafo.

Generalmente, un cammino P all'interno di un grafo è così definito:

$P = < U_0, \dots ,U_k > : \forall i : 0 \le i \le k : Ui \in V, \forall i : 0 \le i \le k - 1 : (Ui, Ui+1) \in E$

La cardinalità di P corrisponde al numero di archi all'interno del cammino.

Un cammino **semplice** è un cammino in cui non si ripete mai lo stesso nodo due volte. Quindi bisogna modificare la definizione come segue:

$P = < U_0, \dots ,U_k > : \forall i : 0 \le i \le k : Ui \in V, \forall i,j : 0 \le i \ne j \le k - 1 : (Ui, Uj) \in E$

Un cammino che parte da Ui e arriva a Ui si definisce cammino **ciclico**.

$P = < U_0, \dots ,U_k > : \forall i : 0 \le i \le k : Ui \in V, \forall i,j : 0 < i \ne j < k - 1 : (Ui, Uj) \in E$

Un grafo si dice **aciclico** quando non ha cicli.

Un grafo si dice connesso quando ogni nodo è connesso con tutti gli altri.

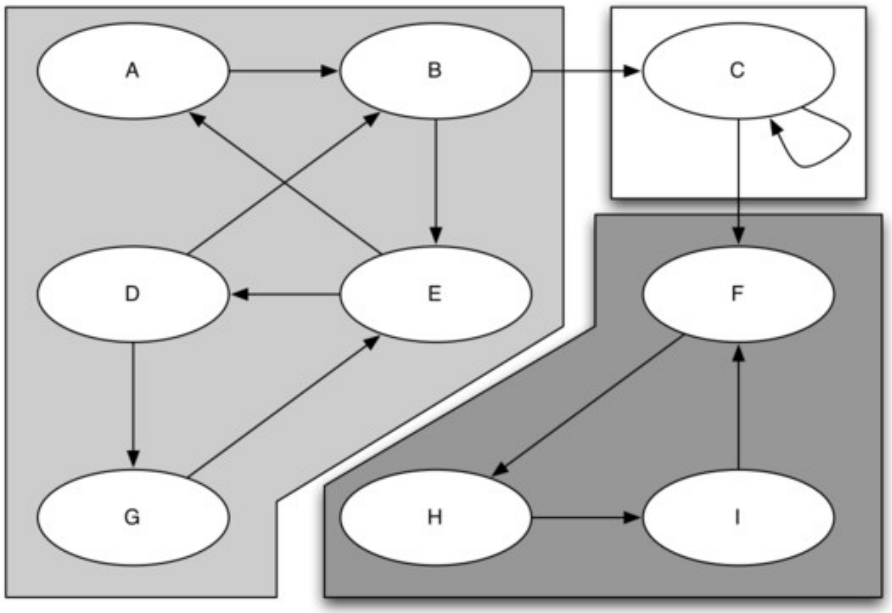
Osservazione: un albero si può vedere come un grafo orientato, aciclico e connesso.

Componenti connesse.

Una **componente connessa** è un sottoinsieme massimale di vertici mutuamente raggiungibili l'uno dall'altro.

Si può dimostrare per assurdo che un nodo non possa essere contenuto in due componenti connesse distinte.

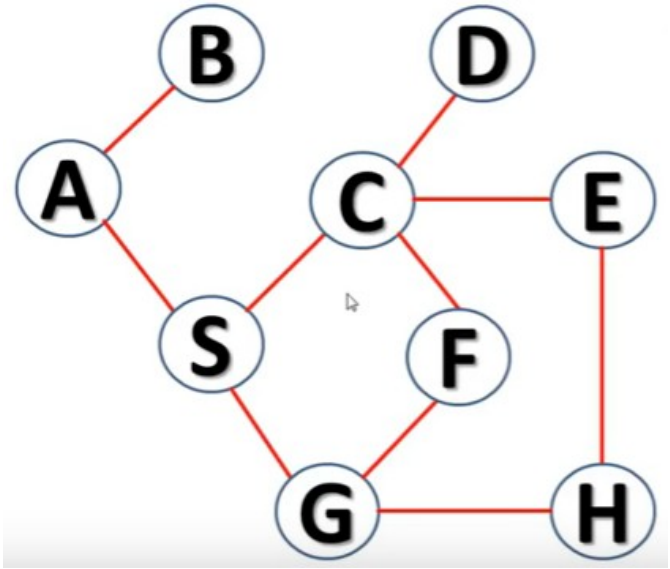
Un grafo orientato contiente solo **componenti fortemente connesse**.



Breadth-first Search (BFS)

La BFS, o in italiano “ricerca in ampiezza”, è un algoritmo di ricerca per grafi che, partendo da un vertice detto **sorgente**, permette di cercare il cammino fino ad un altro nodo scelto e connesso al nodo sorgente.

Si ipotizzi una coda Q ed un grafo G.



L’algoritmo comincia il puntando al nodo sorgente A.

Si effettua una scansione dei nodi adiacenti ad A ovvero B ed S, essi vengono aggiunti nella coda Q ed A viene marchiato come visitato e stampato in output.

Si punta adesso all’elemento uscente dalla coda, ovvero B, si esegue la scansione e nulla viene aggiunto. B viene marchiato come visitato e stampato.

Si punta adesso ad S, che viene espulso dalla coda. Dalla scansione si accodano i nodi C e G. S viene marchiato come visitato e viene stampato.

E così via..

Durante l’algoritmo si possono raccogliere dati come i nodi precedenti durante il cammino, la distanza di ogni nodo dal nodo sorgente etc.

La visita finisce quando la coda è vuota.

La complessità della BFS risulta essere O(E+V), ovvero O(E) per la visita degli archi, sommato ad O(V) per la coda.

```
void bfs(T value) {

    int s = keys->getIndex( value );
    Queue<int> *queue = new Queue<int>(V);
    for(int i = 0; i < V; i++) {
        distance[i] = -1;
        parent[i] = -1;
        color[i] = WHITE;
    }

    distance[s] = 0;
    color[s] = GREY;
    queue->enqueue(s);

    while(!queue->isEmpty()) {
        int e = queue->dequeue();
        color[e] = GREY;

        for(int i = 0; i < V; i++) {
            if( edges[e][i] && color[i] == WHITE ){

                queue->enqueue( i );
                distance[i] = distance[e] + 1;
                parent[i] = e;
                color[i] = GREY;

            }
        }
        color[e] = BLACK;
    }
}
```

Depth-first Search (DFS)

La DFS, in italiano ricerca in profondità, è un algoritmo di ricerca che, a differenza della ricerca in ampiezza, ha la caratteristica di essere intrinsecamente ricorsivo.

La strategia di ricerca esplora il grafo andando, in ogni istante dell'esecuzione dell'algoritmo, il più possibile in profondità: gli archi del grafo vengono esplorati a partire dall'ultimo vertice scoperto v che abbia ancora degli archi non esplorati uscenti da esso. Una volta terminata l'esplorazione di tutti gli archi non esplorati del vertice v, si ritorna indietro per esplorare tutti gli archi uscenti a partire dal vertice da cui v' era stato precedentemente scoperto. Il processo di esplorazione continua fin quando tutti i vertici del grafo non siano stati esplorati. Il sottografo dei predecessori, che viene generato dalla visita in profondità può essere costituito da più alberi: tale sottografo è definito foresta DFS, composta, quindi, da diversi alberi DFS. La ricerca DFS, oltre a generare la foresta DFS marca ogni vertice con ben precise informazioni temporali, in particolar modo aggiorna due etichette per ogni vertice: d[v] che registra quando il generico vertice v è stato scoperto ed f[v] che registra quando è stata esplorata l'intera lista di adiacenza di v (ovvero tutti i vertici che sono raggiungibili a partire da esso).

I vertici vengono colorati in modo differente a seconda dei casi: WHITE, colore che ogni vertice assume prima del tempo d[u], GREY tra il tempo v[u] ed il tempo f[u] e BLACK nel seguito. Per ogni vertice u vale inoltre d[u] < f[u]. Segue l'implementazione:

```
void dfsVisit(int i) {
    color[i] = GREY;
    start[i] = time++;

    for(int j = 0; j < nv; j++) {
        if( edges[i][j] && color[j] == WHITE ) {
            dfsVisit(j);
        }
    }

    color[i] = BLACK;
    end[i] = time++;
}

void dfs() {
    time = 0;
    for(int i = 0; i < V; i++) {
        start[i] = -1;
        end[i] = -1;
        color[i] = WHITE;
    }

    for(int i = 0; i < nv; i++) {
        if( color[i] == WHITE )
            dfsVisit(i);
    }
}
```

Anche in questo caso, la complessità risulta essere O(V+E).

Ordinamento topologico.

In teoria dei grafi un ordinamento topologico è un ordinamento lineare di tutti i vertici di grafo. I nodi di un grafo si definiscono ordinati topologicamente se sono disposti in modo tale che ogni nodo venga prima di tutti i nodi ad esso adiacenti. L'ordinamento topologico non è un ordinamento totale, poiché la soluzione può ***non essere unica****. È possibile ordinare topologicamente un grafo se e solo se non ha cicli, cioè solo se è un grafo aciclico diretto (DAG).*

Si può dimostrare che invertendo il risultato di una DFS in un grafo aciclico si ottiene un ordinamento topologico del grafo stesso:

Nella DFS si stampano ricorsivamente gli elementi adiacenti, mentre nell'ordinamento topologico si vuole ottenere una stampa del nodo e dei suoi adiacenti, poi gli adiacenti degli adiacenti e così via.

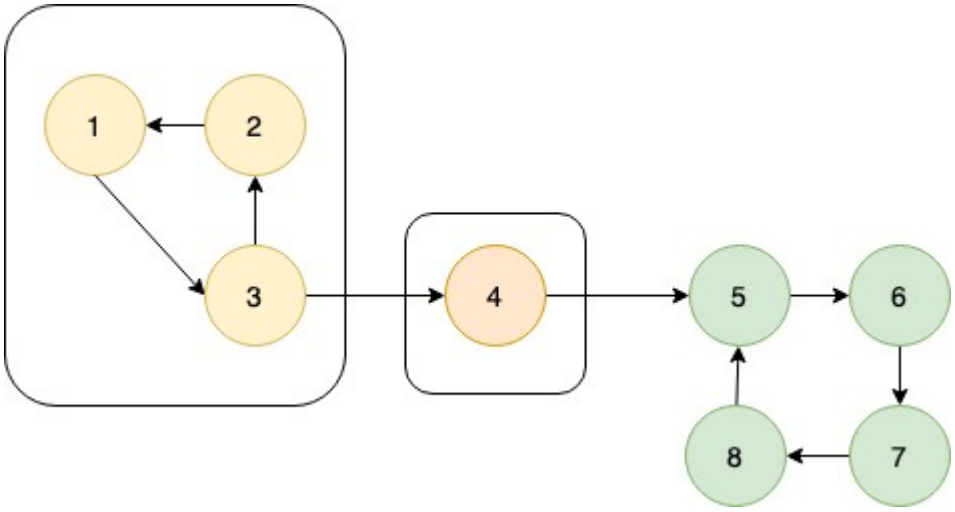
La soluzione sta nell'utilizzare uno stack temporaneo durante la dfs e impilare gli elementi in chiusura (quando il colore del nodo diventa nero), per poi stampare tutto lo stack a fine visita.

```
void dfsVisit(int i) {
    color[i] = GREY;
    start[i] = time++;

    for(int j = 0; j < nv; j++) {
        if( edges[i][j] && color[j] == WHITE ) {
            dfsVisit(j);
        }
    }

    color[i] = BLACK;
    end[i] = time++;
    stack->push( keys->getKey(i) );
}

void dfs() {
    time = 0;
    while(!stack->isEmpty()) stack->pop();
    for(int i = 0; i < V; i++) {
        start[i] = -1;
        end[i] = -1;
        color[i] = WHITE;
    }

    for(int i = 0; i < nv; i++) {
        if( color[i] == WHITE )
            dfsVisit(i);
    }
}

void sort() {
    dfs();
    while(!stack->isEmpty())
        cout << stack->pop() << " ";
    cout << endl;
}
```

Algoritmo di Tarjan per le componenti fortemente connesse.

L’algoritmo di Tarjan è un algoritmo usato per trovare le componenti fortemente connesse di un grafo.

L'idea alla base dell'algoritmo è che una DFS inizia sempre da un nodo di partenza. Le componenti fortemente connesse, insieme, formano i sotto-alberi dell'albero di ricerca le cui radici sono le componenti fortemente connesse.

I nodi sono dunque inseriti in uno stack, ossia una pila, nell'ordine in cui sono visitati. Quando la ricerca risale da un sotto-albero visitato, i nodi corrispondenti sono ripresi dalla pila e viene determinato se ognuno è la radice di una componente fortemente connessa. Se lo è un nodo, allora insieme a tutti quelli presi prima (cioè appartenenti al sotto-albero di cui è radice) formano una componente fortemente connessa.

Il punto centrale dell'algoritmo consiste nel determinare se un nodo è la radice di una componente fortemente connessa. Per farlo, a ogni nodo v è assegnato un indice di profondità della ricerca, chiamato v.indice, che viene incrementato man mano che vengono trovati nuovi nodi. Inoltre, ogni nodo ha un altro valore assegnato chiamato v.minima_distanza (o lowlink) che indica che indice deve avere per essere raggiungibile dalla radice. Quindi, un nodo è la radice di una componente fortemente connessa se e solo se indice e minima_distanza sono uguali. Il valore minima_distanza è calcolato durante la ricerca depth-first in modo da poter essere sempre recuperabile.